OpsGate

Technical Documentation

Architecture, authentication, tool reference, and security model

Version 7.1 | September 2026

OpsGate is a multi-tenant MCP (Model Context Protocol) server that governs how AI coding assistants — Claude and Replit Agents — are allowed to plan and implement changes on a real codebase. It compiles plain-language requests into self-contained, gate-checked implementation prompts, enforces deterministic scope/capability/path rules before any work happens, and carries state across Replit's disposable per-session model so a multi-phase task can resume correctly. This document is the complete technical reference: architecture, the tenant/authentication model, the full tool inventory, the governed prompt-compiler chain, and the security properties this system actually guarantees (and where it deliberately does not).

1. System Overview

- **Claude** — turns a plain-language request into a routed, gate-checked, self-contained implementation prompt.
- **A Replit Agent** — executes that prompt inside the actual project and reports back what it did.
- **Neither agent defines or enforces the rules itself** — both check against the same OpsGate server, so the rules exist in exactly one place.

Why the split exists

- **A Replit session has no memory between prompts** — each one is a fresh session with no knowledge of any earlier one.
- **Claude tracks state across that gap** — using OpsGate's own persisted run state (Section 4), not its own conversation memory, which would not survive a new Replit session either.

What OpsGate is not

- **It does not write code.** — Every tool it exposes is a deterministic check, a text-compilation step, or a state read/write — never an implementation action.
- **It is not a general-purpose MCP framework.** — It is one purpose-built server for one governance problem: keeping AI-assisted changes to a real codebase inside agreed, auditable boundaries.
- **The two-mount split (Section 2) is a discoverability convenience, not a security boundary.** — Both mounts share identical authentication and tenant resolution; the gates inside each tool call are what actually enforce anything.

Multi-tenancy

- **OpsGate is one hosted service multiple projects (“tenants”) connect to** — not one deployment per project.
- **Each tenant has its own resolved profile** — frontend/backend write roots, extra protected paths, an optional business-context file reference, and its own credentials.
- **A tenant can never see or affect another tenant's profile, tokens, run state, or audit history.**
- **Every tenant-scoped function resolves its tenant identity purely from the caller's own authenticated token** — never from a caller-suppliable field.

2. Architecture

One Python process runs a single Starlette ASGI application with two mounted sub-applications, each its own FastMCP instance exposing only the tools that role needs.

Request flow

- **Claude and a Replit Agent each connect over HTTPS with a token** — to the OpsGate MCP server (one process, one Starlette app).
- **Claude's calls go through /mcp/claude/** — Replit's go through /mcp/replit/ — both pass through the identical TokenAuthMiddleware.
- **Every call resolves tenant identity against tenants/registry.json** — reads/writes run state and decisions under runs/<tenant_id>/, and reads knowledge resources from content/**.

Two mounts, one auth boundary

- **/mcp/claude/ exposes 25 tools** — 14 shared, 11 exclusive to the prompt-compiler chain and run recovery.
- **/mcp/replit/ exposes 16 tools** — the same 14 shared tools, plus 2 exclusive to instruction sync.
- **Both sit behind the identical TokenAuthMiddleware** — a valid token authenticates identically against either path.
- **The split exists purely so each caller sees a shorter, more accurate tool list for its own role** — it is explicitly documented in code as not a security boundary.

Stateless HTTP transport

- **Both FastMCP instances are constructed with stateless_http=True.**
- **Every incoming request gets its own fresh transport and task** — with no persistent Mcp-Session-Id concept at all.
- **This is a deliberate, security-motivated choice, not a default** — see Section 9 for why stateful mode was actively unsafe here.

Where state actually lives

- **Nothing needed across calls lives in server memory.**
- **Everything persists to disk under OpsGate's own repository:** — the tenant registry (tenants/registry.json), per-tenant run state and logs (runs/<tenant_id>/), and the read-only instruction/spec content every tenant shares (content/**).
- **This is what allows the stateless-HTTP transport to cost nothing functionally** — no tool in this system ever depended on in-memory session continuity to begin with.

3. Tenant & Authentication Model

Tenant registry

- **tools/opsgate_tenants.py is the single profile-resolution mechanism** — the CLI and the MCP server both resolve through it, with no per-project hardcoded profile and no external config file to walk.
- **Storage is a file-backed JSON registry** — tenants/registry.json, gitignored, self-healing to 0600/0700 permissions on every save, exclusively locked via a companion .lock file for every read-modify-write

cycle.

- **A caller with no specific tenant identity resolves to local-dev** — a built-in default with unset write roots rather than a guessed one.

Tokens

- **Generated via secrets.token_urlsafe(32)** — only the SHA-256 hash is ever stored, never the plaintext — a leaked registry file does not itself leak usable credentials.
- **Every hash comparison in the resolution and revocation paths uses hmac.compare_digest** — not a plain equality check.
- **A token carries an admin boolean and an optional free-text label** — operator-facing only, never used for authorization — minted per-consumer so revoking one consumer's access never silently revokes another's.
- **A tenant can hold multiple tokens simultaneously** — e.g. one for Claude, one for Replit, one labeled per environment — each independently revocable.

Resolution and the admin-override path

- **resolve_tenant_from_token() returns (tenant_id, is_admin)** — for a valid, non-revoked token, or (None, False) otherwise — it always fails closed, never falling back to a default tenant.
- **A separate resolve_tenant(token, override_tenant_id) function** — supports an admin token addressing a different tenant's profile, but only when the token resolves as admin and the override ID names a real tenant; a non-admin token can never address another tenant's profile this way.
- **This override path has no caller anywhere in the live MCP server today** — documented as an open, deliberately-unused capability, not a gap in what's actually reachable.

Two authentication transports, one resolution path

- **TokenAuthMiddleware accepts either the X-Opsgate-Token header (checked first)** — or an Authorization: Bearer <token> header (the transport OAuth mandates) — both resolved through the identical tenant lookup.
- **If neither resolves to a real tenant** — the request falls back to a single shared-secret comparison (OPSGATE_MCP_TOKEN), which resolves to the local-dev identity.
- **A request matching neither path is rejected with 401** — and a WWW-Authenticate header pointing at OAuth discovery metadata.

OAuth 2.1 + PKCE wrapper

- **mcp-server/opsgate_oauth.py is a minimal, purpose-built OAuth authorization server** — not a general-purpose one — added specifically because Claude's org-level Connector UI is OAuth-only with no static-header option.

- **Its /token endpoint mints nothing new** — a PKCE-verified code exchange hands back a single pre-configured backing token as the OAuth access token, so tenant resolution downstream is completely unaffected by this wrapper — it is a transport adapter, not a second identity system.
- **Covered:** — discovery metadata (/.well-known/oauth-authorization-server, /.well-known/oauth-protected-resource), PKCE code_challenge/verifier matching, single-use authorization codes, a pinned allowed redirect_uri, and client_id/secret verification.

4. The Governed Prompt-Compiler Chain

This is the numbered sequence Claude follows to turn a request into governed Replit work and carry the result back into the next phase. It is documented live to every connected Claude session via the opsgate://knowledge/claude-mcp-workflow resource, so the operating manual itself never drifts out of sync with the server that serves it.

intake_request text → request	**route_request** resolve mode + skill	**preflight** scope + capability gates	**compile_prompt** self-contained prompt

↓ hand off to Replit ↓

Replit executes the compiled phase	**parse_report** structured evidence	**next_phase_prompt** next phase, or Blocked

next_phase_prompt's output becomes the next compile_prompt call — repeated until the run completes or a phase reports blocked.

- **1. opsgate_intake_request** — Plain-language text → a draft structured request (deliverable, outcome, module, likely authorizations), scored via a word-aware lexical matcher — not naive substring matching, so “review” inside “preview” doesn't false-match, and a genuinely ambiguous outcome is flagged rather than silently resolved to whichever deliverable happened to be checked first.
- **2. opsgate_route_request** — Resolves deliverable, internal mode, skill, required references, execution shape (bounded vs. phased), and capability — purely from the resolved routing manifest and the caller's own tenant profile, never from anything else.
- **3. opsgate_preflight** — Runs the scope, capability, and protected-path gates together, before any prompt is compiled. A missing authorization or a protected-path violation must block here — never surface later inside Replit's own session.
- **4. opsgate_compile_prompt** — Produces the literal, self-contained prompt text to hand to Replit as a new session. Every caller-supplied field is fenced or inline-sanitized before being embedded (Section 9).
- **5. opsgate_init_run** — For phased work only: persists the request, resolved route, initial gate result, and handoff state to runs/<tenant_id>/<request_id>/ — this is what survives across Replit's disposable sessions, not anything held in this conversation.
- **6. Hand off to Replit** — The compiled prompt is delivered as a new Replit session's task.

- **7. opsgate_parse_report** — Replit's final report text → structured fields: outcome, PASSED/FAILED/NOT RUN checks, files changed, HITL decisions, blockers, residual risk. A report with no recognizable structure is flagged (has_signal: false) rather than silently treated as a clean pass.
- **8. opsgate_next_phase_prompt** — Given the current run state and the parsed report, emits the next phase's prompt — or refuses outright ("Phase Blocked" / "No Next Phase") if the run's own status is blocked or the report was unparseable, regardless of what any individual phase's status looks like.

Bounded vs. phased execution

- **Bounded** — the outcome can be safely inspected, implemented, verified, and rolled back together in one batch.
- **Phased** — the request involves multiple independently reversible surfaces, schema/migration/seeding work, authentication or public-contract changes, broad refactors, or anything that cannot be verified and rolled back as one unit.
- **Only phased runs get persisted state via opsgate_init_run** — a phase prompt only ever grants authority for that one phase — prior reports are evidence, never standing authority for what comes next.

5. Capability Gates & Protected Paths

Every capability has a declared default posture and a list of preconditions it requires regardless:

Capability	Default posture	Requires (evidence needed to proceed)
ordinary_application_change	allowed_when_scoped	observable outcome, exact scope inside normal write paths
audit_or_diagnosis_without_fixes	read_only	approved inspection scope
broad_architecture_refactor	blocked	explicit broad outcome, named approved source tree, preserved contracts, phased rollback
instruction_maintenance	blocked	explicit instruction-change request, exact instruction paths
schema_migration_backfill	blocked	explicit request, approved target mapping, named paths, safe non-prod environment, rollback plan
data_seeding	blocked	explicit request, named seed paths, safe non-prod environment, approved profiles/scale, idempotency
contract_change_or_destructive_cleanup	blocked	explicit changed boundary, consumer evidence, compatibility plan, recovery plan
package_config_environment_deployment	blocked	exact explicit authorization, task-specific safety prerequisites

- **A capability whose default posture is not blocked is authorized by default** — an explicit authorized: false on one of those must not block it either.
- **Only a blocked-default capability requires an explicit authorized: true to proceed.**
- **This single rule — capability_authorized() in tools/opsgate_routing.py — is shared by every tool that makes a capability decision** — specifically so they can never disagree about the same request.

Protected paths

- **Every tenant gets a universal baseline regardless of its own configuration** — version control internals, secrets files, dependency trees, CI configuration, agent memory — merged with that tenant's own extra_never_access list and its resolved frontend/backend write roots.
- **A path that may resolve to protected content is treated as protected outright** — there is no partial-match exception.

6. The Human-in-the-Loop Protocol

OpsGate deliberately separates two kinds of gate failure, because only one of them is actually a decision for a human to make:

Kind	Meaning	Response
Deterministic	scope_gate, capability_gate, protected_path_gate — one correct answer, no judgment involved	Name the exact failed gate and what's missing. Stop. Never phrased as a decision request.
Judgment (HITL)	One of exactly three cases below — genuine ambiguity only discoverable during real work	Pause the entire task; return a structured decision request; wait for a human reply.

The three HITL cases — and only these three

- **Unknown next step** — even after bounded, approved inspection, the next step genuinely cannot be determined.
- **Tied valid options** — two materially correct answers exist and nothing in evidence, convention, or existing rules favors one.
- **Self-made scope expansion** — proceeding would require the agent to decide, on its own, to expand the approved scope.

Complexity, risk, or being security-related is explicitly not a HITL trigger on its own — if a deterministic rule already permits, requires, or forbids the action, that rule is followed directly, with no question asked.

The decision-request format

> **# HITL decision required**
> ID: HITL-task-Pphase-Qnumber Blocked check: Question:
> Evidence checked: Options: A. – B. –
> Exact resume point: Required reply: DECIDE <A|B>

- **A human's reply is persisted outside the conversation via opsgate_record_decision** — append-only, tenant-scoped, hitl_id shape-validated against the same pattern a real HITL object requires — necessary because neither this conversation nor whichever Replit session eventually resumes the work will otherwise have any record of it.

Per-Action Gate vs. Mandatory HITL Gate

- **The full Mandatory HITL Gate evidence table runs once before editing, once before each phase, and once before the final report.**
- **A lighter Per-Action Gate** — the same three Known/Single/Bounded checks, at finer grain — runs before every individual state-changing action in between, so an agent can't drift out of bounds between checkpoints without it being caught immediately.

7. Tool Inventory

16 tools on /mcp/replit/, 25 on /mcp/claude/, 14 shared between both. Every tool resolves its tenant identity purely from the authenticated caller — never from a request field — and a deterministic gate tool returning can_proceed: false is a normal, valid result, not a protocol-level error.

Shared gate, profile, and decision tools (both mounts)

Tool	Purpose	Notes
opsgate_show_profile	Resolved tenant profile, write roots, and protected paths.	No request required
opsgate_check_capability	Standalone deterministic capability_gate check.	Shares capability_authorized()
opsgate_check_paths	Standalone deterministic protected_path_gate check.	Same matcher as preflight
opsgate_preflight	Scope + capability + protected-path gates, together.	Run before every phase
opsgate_record_decision	Appends one HITL decision to the tenant's decisions.pylog.	hitl_id shape-validated

Shared self-service and observability tools (both mounts)

Tool	Purpose	Notes
opsgate_list_own_tokens	Label/admin metadata for the caller's own tenant's tokens.	Never returns token values
opsgate_issue_own_token	Mints a new token for the caller's own tenant.	Always non-admin; shown once
opsgate_revoke_own_token	Revokes a token, only if it belongs to the caller's own tenant.	Ownership-checked
opsgate_list_audit_log	The caller's own tenant's recent tool-call history.	Filtered from a shared log file
opsgate_quota_usage	Call volume over 1h/24h/7d windows, per tool.	Visibility only, no enforcement

Shared admin-gated tools (both mounts, require an admin token)

Tool	Purpose	Notes
opsgate_admin_create_tenant	Registers a brand-new tenant.	Fails if the ID already exists
opsgate_admin_list_tenants	Every tenant's public profile, registry-wide.	Never exposes token hashes
opsgate_admin_issue_token	Mints a token for any tenant, optionally admin.	Can mint further admin tokens
opsgate_admin_revoke_token	Revokes any token, regardless of owning tenant.	No ownership check needed

Replit-only tools

Tool	Purpose	Notes
opsgate_sync_instructions	Manifest ({path, size}) of every instruction/skill file.	No content, size-capped
opsgate_sync_file	Full content for one path from that manifest.	Fetched one at a time

Claude-only tools

Tool	Purpose	Notes
opsgate_intake_request	Plain text → draft structured request.	Word-aware lexical scoring
opsgate_route_request	Resolves mode, skill, references, capability.	Pure function of request + tenant
opsgate_init_run	Persists request/route/gate/handoff for a phased run.	Writes to disk; size-capped
opsgate_list_runs	The caller's own tenant's tracked runs.	Most recent first, capped
opsgate_get_run	Full persisted state for one of the caller's own runs.	Raises on a corrupted file
opsgate_compile_prompt	The compiled prompt for a request/route pair.	Prose text, not JSON
opsgate_next_phase_prompt	Next phase's prompt, or a refusal.	Refuses on any blocked state
opsgate_parse_report	Replit's report text → structured fields.	Flags unparseable input
opsgate_lint_report	Confirms a report states every required section.	Pre-check before parsing
opsgate_lint_prompt	Confirms a compiled prompt states every required concept.	Accepts multiple phrasings
opsgate_export_ruleset	One snapshot of every knowledge resource below.	For offline/CI use

Deliberately not exposed as tools: validate-json, validate-engine, test-all — self-test commands run by hand inside this repository, not gate/routing calls a live task needs. Tenant deletion and real rate-limit enforcement are likewise deliberately unbuilt as of this writing (Section 9).

8. Knowledge Resources

Read-only governance content, backed by canonical source files read live at call time — never a copied string, so a resource can never drift out of sync with its own source.

Resource	Always-on?	Backed by
opsgate://knowledge/hitl-protocol	Yes, both mounts	The full HITL specification, unabridged
opsgate://knowledge/security-rules	Yes, both mounts	Durable security rules, scaffolding stripped

Resource	Always-on?	Backed by
opsgate://knowledge/claude-mcp-workflow	Yes, Claude only	The numbered chain in Section 4, as served live
opsgate://knowledge/skill-workflow/{skill}	Route-conditional	One skill's own numbered workflow
opsgate://knowledge/instruction-object/{name}	Route-conditional	One domain's durable rules (backend, frontend, etc.)

9. Security Model

Critical fix: stateless HTTP transport — session identity could not be trusted in stateful mode

In the MCP SDK's default stateful mode, a session's tool calls execute inside a task spawned once at session-creation time. Python's contextvars do not propagate across tasks, so a later request's own authenticated identity never reached that task — and the SDK's built-in session-owner guard never activated here, since it requires an auth provider this server does not use.

Net effect: any valid token could resume any other tenant's — or an admin's — existing session by presenting its session ID.

Verified exploitable against a real running instance, then closed by switching both FastMCP instances to stateless_http=True, which removes the persistent-session mechanism entirely.

A permanent regression test sends a request with a fabricated session ID and asserts it resolves strictly to the requesting token's own tenant.

Caller-supplied text is never trusted as instruction

- **Free-text fields (outcome, must_not_change, acceptance) can originate from a non-technical user's plain language via opsgate_intake_request** — with no review before reaching a compiled prompt.
- **Each is wrapped in a fixed delimiter pair** — <<<CALLER_SUPPLIED_DATA>>> / <<<END_CALLER_SUPPLIED_DATA>>>, with the delimiter itself neutralized if it appears literally inside the caller's own text, so a field cannot forge its own closing marker.
- **Inline fields embedded outside a fenced block (a title, a table cell)** — are flattened to one line and stripped of the same delimiters via sanitize_inline_text(), closing an injection path a prior audit found: a route with no configured capability could fall back to a raw, caller-supplied authorizations dict key, embedded unsanitized into the compiled prompt's routing table.
- **This is a real, structural mitigation, not a complete one** — no prompt-injection defense is airtight — but every hard boundary (protected paths, capability gates) is enforced by deterministic tool calls regardless of what any compiled prompt says.

Tenant isolation

- **Every tenant-scoped function resolves its tenant purely from the caller's own authenticated token** — never from a request field.
- **Independently re-verified function by function, not just asserted** — run storage, audit-log filtering, token operations, and the admin-gated tools all scope correctly, confirmed by a dedicated cross-tenant adversarial audit and by two real tenants exercised against the live running server in tests/test_opsgate_mcp_integration.py.

Fail-closed by construction

- **An unknown, malformed, or revoked token is rejected immediately** — never falls back to a default identity.
- **A malformed (non-object) authorization entry is treated as not-authorized** — rather than crashing or silently passing.
- **A corrupted run-state file is surfaced as a named, explicit error** — from the single-run recovery path, rather than either crashing unhelpfully or returning data indistinguishable from "never written."
- **Run-state files are read via ast.literal_eval, not exec()/import** — a reader exposed to an MCP caller does not execute file content as code, even though the writer only ever produces safe literals today.

Bounded input, bounded output

- **Request size is capped** — 50,000 characters for opsgate_init_run; decision fields are capped at 5,000 characters.
- **List/log-reading tools cap the number of items a single call can return (200)** — opsgate_list_runs, opsgate_list_audit_log, regardless of what a caller asks for — including correctly treating a requested limit of exactly zero as zero results, not the default page size, and rejecting a negative limit outright rather than silently mis-slicing.

Structured, tenant-attributed audit logging

- **Every tool call — successful or not — is wrapped once at registration time** — and appended to runs/audit.jsonl with tenant, tool name, success, duration, and (on failure) an error message.
- **This is what actually distinguishes "which tool did this tenant call"** — from the web server's own access log, which only ever shows a generic POST /mcp/claude/ 200 OK regardless of which tool ran inside that request.

Deliberately not built

- **Tenant deletion over MCP was left out on purpose** — it is the single most destructive, least reversible admin operation available, and exposing it needed a deliberate decision rather than a default inclusion alongside the other admin tools.
- **Real rate-limit enforcement was also left out** — there is no existing usage-tracking baseline to set a real numeric threshold against yet, so opsgate_quota_usage ships as visibility only, intended to inform a

real limit once real usage data exists rather than enforcing an arbitrary one now.

10. Deployment & Operations

As of this writing, OpsGate runs as a single supervised process on one machine (not yet on dedicated server infrastructure) — a deliberate, acknowledged interim state, not a target architecture. A container packaging of the same server exists and is verified (Section 12), but it is not yet what serves production traffic.

Aspect	Current state
Process supervision	launchd (com.opsgate.mcpserver) — auto-restarts on crash, replacing manual process management
Public exposure	Tailscale Funnel, terminating TLS in front of the local process
Health check	Unauthenticated GET /health — confirms the tenant registry itself parses, not just that the process answers
Audit log	runs/audit.jsonl, append-only, structured per-tool-call entries
Logs	~/Library/Logs/opsgate-mcp-server.log (stdout/stderr under launchd)
Bind host	127.0.0.1 only — reachable externally only through the Funnel

- **A code change requires restarting the supervised process.**
- **A content change under content/** is picked up on the next call with no restart** — since those files are read from disk live rather than cached.
- **Every restart severs any in-flight MCP session, and a connected client must reconnect** — though with stateless HTTP now in place, there is no session to sever in the first place going forward.

11. Setup Process: Connecting a New Tenant

This section walks through standing up OpsGate for a new project end to end: provisioning the tenant, minting tokens, and connecting both the Claude and Replit sides. It draws entirely on the tenant/authentication model (Section 3), the tool inventory (Section 7), and the deployment model (Section 10) above — nothing here introduces a new mechanism.

1. Prerequisites

- **The OpsGate server is running and healthy.** — GET /health returns healthy, confirming the process is up and the tenant registry parses.
- **An admin token exists.** — Admin tokens are the only credential that can create a new tenant or mint a token on another tenant's behalf (opsgate_admin_create_tenant, opsgate_admin_issue_token).

- **The project's write boundaries are decided up front.** — Which paths are the normal working area (frontend/backend write roots), and which paths — beyond the universal protected baseline — must never be touched (extra_never_access).

2. Register the tenant

- **Call opsgate_admin_create_tenant with an admin token.** — Supply the tenant ID, the resolved write roots, any extra protected paths, and an optional business-context file reference. The call fails if the ID already exists, so it is safe to retry against a checked ID.
- **The tenant is written to tenants/registry.json.** — Self-healing to 0600/0700 permissions on save, exclusively locked for the write — no manual file editing is required or supported.

3. Issue tokens for each consumer

- **Mint one token per consumer, not one shared token.** — A typical tenant holds at least two: one for its Claude connection and one for its Replit connection — each independently revocable without affecting the other.
- **Use opsgate_admin_issue_token (admin-gated) for the initial tokens,** — or opsgate_issue_own_token once the tenant already holds one token and wants to self-service the rest.
- **Only the SHA-256 hash is ever stored.** — The plaintext token is shown exactly once, at mint time — record it immediately; it cannot be retrieved again, only revoked and replaced.

4. Connect the Replit side

- **Point Replit's MCP connection at the /mcp/replit/ mount** — on the OpsGate host.
- **Authenticate with the Replit-side token** — via the X-Opsgate-Token header, or as an Authorization: Bearer token if the client only supports the OAuth-style transport.
- **Confirm the connection with opsgate_show_profile.** — A successful call confirming the expected write roots and protected paths is the signal the tenant resolved correctly — not just that the request returned 200.

5. Connect the Claude side

- **For a direct token-header client, connect at /mcp/claude/** — the same way as Replit, with the Claude-side token.
- **For Claude's org-level Connector UI (OAuth-only)** — use the OAuth 2.1 + PKCE wrapper (Section 3): configure the connector against the discovery metadata at /.well-known/oauth-authorization-server, with the pinned redirect_uri and client credentials for this deployment. The wrapper exchanges the OAuth flow for the same pre-configured backing token — tenant resolution is unaffected by which transport was used to present it.
- **Confirm with opsgate_show_profile from the Claude side as well.** — Both mounts should resolve to the same tenant profile.

6. Verify end to end

- **Run a single opsgate_preflight call for a small, ordinary_application_change request.** — A can_proceed: true result confirms the scope, capability, and protected-path gates are all resolving against the new tenant's real profile, not a default.
- **Check opsgate_list_audit_log** — to confirm the calls made during setup are attributed to the new tenant, not local-dev.

Ongoing maintenance

- **Rotating or revoking a token is self-service** — opsgate_issue_own_token / opsgate_revoke_own_token — no admin call needed once the tenant is provisioned.
- **Changing write roots or the protected-path list** — is a registry update, not a re-provisioning — it does not require new tokens.
- **A code change to the server requires a process restart; a content/** change (templates, instructions, skills) does not.** — See Section 10.

12. Recommended Approach: Migrating Off the Current Hosting

This section documents the recommended approach for moving OpsGate off a personal machine and a tunneling service onto real, durable server infrastructure. The deployment unit for that move — a container image and a compose stack — is built, verified, and committed; the hosting move itself has not happened. What follows is why the move is required, what the container provides, what any host must guarantee, the realistic hosting shapes, and the cutover.

Why this move is required

- **Every connected project depends on one machine.** — The server runs as a single process on one personal Mac. When that machine is off, asleep, offline, or being rebuilt, every tenant's gate checks, prompt compilation, and run recovery stop — there is no second instance and no failover.
- **The public endpoint depends on a personal Tailscale account and a Funnel on that same machine.** — If Tailscale is stopped or the Funnel configuration is lost, the public hostname stops resolving and every client loses OpsGate — while the server process itself stays perfectly healthy on loopback, so nothing on the server side registers a fault.
- **Nothing watches the public endpoint.** — GET /health is only checked when a person looks. There is no external monitoring or alerting, so an outage is discovered when a Replit or Claude session fails, not before.
- **This has already happened.** — On 2026-09-06 Tailscale was found stopped on the host with no serve configuration; the server was healthy on 127.0.0.1:8765 the entire time, the public hostname returned NXDOMAIN, every connected client was cut off, and no alert fired.

- **There is no handover.** — The launchd configuration, the Funnel setup, and the operating knowledge live on one person's machine. The Funnel command itself is not recorded anywhere in the repository, so nobody else can recreate the public endpoint from the checkout alone.
- **Consequence:** — the current arrangement is acceptable only as a short-lived interim state. Moving to server infrastructure with durable storage, a stable domain, a restart policy, and external health monitoring is a requirement for running OpsGate as a shared service — not an optimization.

The deployment unit: the container

- **Dockerfile, docker-compose.yml, and .dockerignore live at the repository root.** — The build context is the repository root, mirrored one-to-one under /app, because the server imports the engine from the sibling tools/, reads content/** live, and persists state under tenants/ and runs/ — the repo layout is the runtime layout.
- **python:3.12-slim base; mcp-server/requirements.txt installed on its own layer** — so a code-only change does not reinstall dependencies.
- **Runs as an unprivileged user (opsgate, uid 10001) that owns the state directories** — so the registry's own 0600/0700 self-healing works on a fresh volume with no manual chown.
- **No secrets and no state in the image.** — .dockerignore excludes mcp-server/.env, tenants/, runs/, .git/, .venv/, tests/, and docs/. Configuration arrives only through the environment (compose reads mcp-server/.env via env_file — the same file and keys the bare-process deployment uses); state lives in two named volumes, opsgate-tenants and opsgate-runs.
- **Binds 0.0.0.0 inside the container (OPSGATE_MCP_HOST)** — a published port cannot reach a loopback-only process. The mcp SDK's Host-header allow-list still applies unchanged, so this does not widen what the server answers to. Compose publishes on 127.0.0.1:8765 only, for a TLS-terminating reverse proxy to front — the same shape as today's 127.0.0.1-plus-Funnel.
- **HEALTHCHECK on GET /health** — so a corrupted tenants/registry.json surfaces as an unhealthy container, not merely a process that answers HTTP.
- **content/** is baked into the image.** — Unlike the bare-process deployment, a content change here is a rebuild — or bind-mount ./content:/app/content:ro to keep it hot-editable.
- **Verified against a running container from empty volumes:** — /health 200; initialize 401 with no token, 200 with one, 421 with a foreign Host header; tenant provisioning via docker exec wrote a 0600 registry; a tenant token and the shared secret resolved to their own identities on opsgate_show_profile, both attributed correctly in runs/audit.jsonl on the volume; the container reported healthy. Operating detail is in mcp-server/README.md, “Docker”.

What has to be true regardless of where the container runs

- **A stable public HTTPS endpoint on a real domain** — not one dependent on a specific machine being powered on and connected to the internet.
- **Durable storage behind the two named volumes.** — This is the single most important requirement: tenants/registry.json and runs/** are plain files, not a managed database, so any host whose volumes

reset on restart, redeploy, or scale-out silently destroys the tenant registry and the entire audit/run history. The volumes must sit on a real persistent disk.

- **Secrets delivered as environment variables, never committed** — the shared-secret token, the OAuth client ID/secret/backing token, the allowed-hosts list, and the pinned OAuth redirect URI — either as mcp-server/.env on the host (what compose reads) or injected from the platform's secret store.
- **A restart-on-failure supervisor.** — Docker's restart: unless-stopped policy plus a daemon that starts on boot replaces launchd; a container platform's own scheduler does the same job.
- **A TLS-terminating reverse proxy in front of the loopback-published port** — with OPSGATE_MCP_ALLOWED_HOSTS set to the proxy's public hostname — otherwise every proxied request is rejected with 421 by the SDK's DNS-rebinding protection.

Three realistic hosting shapes

Option	What it looks like	Tradeoff
A. Single small VM running Docker	A cloud VM with a persistent disk, Docker Engine, the compose stack from this repository, and a reverse proxy (e.g. Caddy, for automatic TLS) on a real domain. Named volumes land on the VM's disk.	Closest to the current setup — swap the Mac for a VM, launchd for Docker's restart policy, the Funnel for a real proxy. Most control, simplest mental model, cheapest to reason about for a single-process, low-traffic service like this. The recommended first target.
B. Container platform with a persistent volume	A platform that runs the image from a registry, manages TLS, the domain, and deploys, and offers an attachable persistent disk mounted at /app/tenants and /app/runs (several mainstream platforms do).	Less operational overhead than a bare VM — no proxy/certificate/daemon to maintain — while still satisfying the durable-storage requirement. A reasonable middle ground, provided the volume is genuinely persistent across redeploys.
C. Stateless/serverless platforms	Container platforms or serverless functions with only ephemeral local storage.	Not recommended without further work first. This would silently lose the tenant registry and audit/run history on every redeploy or scale event unless the storage layer is re-architected onto a real database or object store beforehand — a materially larger change than a hosting move, called out here so it is not picked by default without understanding that cost.

What changes for already-connected clients

- **Every tenant currently connected points at the current Tailscale Funnel hostname** — Replit's “Connect via MCP” URL, and any Claude org-level Connector configuration.
- **Moving to new infrastructure means updating those URLs and the OAuth connector configuration for every connected consumer** — not just standing up a new backend — this is a coordinated cutover step, not a transparent one.

- **The OAuth wrapper's pinned redirect URI, issuer base URL, and allowed-hosts configuration also need updating** — to the new domain as part of the same cutover — all three are environment variables in mcp-server/.env.

Migration checklist

- Provision the host (option A or B above) with Docker and a real, persistent volume; point a real domain at it and put a TLS-terminating reverse proxy in front.
- Clone the repository on the host (or push the image to a registry the platform pulls from) and create mcp-server/.env with the current values plus the new domain in OPSGATE_MCP_ALLOWED_HOSTS and the OAuth redirect/issuer settings.
- docker compose up -d --build, then confirm GET /health returns ok and the container reports healthy.
- Copy the current tenants/registry.json and runs/** into the volumes (docker compose cp into the running container, then chmod 600 the registry) — this is the one true state migration; everything else is a fresh code deploy.
- Confirm an existing tenant token still resolves: an initialize call with that token returns 200, and opsgate_show_profile returns that tenant's own write roots.
- Update every connected tenant's MCP URL and OAuth connector configuration to the new domain.
- Decommission the current Mac-hosted launchd process only after every consumer is confirmed working against the new host — not before.

13. Content Governance (Templates)

- **Two full authoring templates define what a spec or business file must contain** — content/templates/SPEC_FILE_PROMPT_TEMPLATE.md (a 16-section implementation-spec structure) and BUSINESS_FILE_PROMPT_TEMPLATE.md (a 14-section, implementation-neutral business-file structure): document control, traceability IDs (REQ-*, NFR-*, BUS-CAP-*, BUS-REQ-*, BUS-RULE-*, DEC-*, OQ-*, REC-*), acceptance criteria, and a verification matrix.
- **Verified directly against 29 real specification files and 21 real business files from a production tenant** — both templates matched the current convention essentially verbatim, including the ID scheme.
- **The templates themselves are authoring references, not something code reads verbatim.** — compile_artifact_prompt() in tools/opsgate_prompts.py compiles a condensed equivalent for the actual prompt handed to an implementing agent.
- **A delta specification — one documenting only what changed against a prior spec — gets a structurally distinct compiled body** — a Summary-of-Changes table, per-item corrected/new/superseded sections, an Acceptance Criteria Addendum — rather than a shortened version of the full-spec body.
- **Detected from the request's own text via the phrase “delta spec”/“delta specification”** — the same phrase this system's own routing signals already use, so detection and routing agree by construction.